RESEARCH flutter_ffi

RESEARCH flutter_ffi

Scope: driving a Rust VPN core from Flutter for a Mullvad-parity self-hosted VPN app. Covers flutter_rust_bridge (frb) v2 vs UniFFI, Rust→Dart event streaming, per-platform Rust staticlib/cdylib, Flutter on Aurora OS and HarmonyOS NEXT, and Riverpod for a VPN status stream. All versions/dates verified 2026-06-25.

---

1. Rust⍰Dart bridge: flutter_rust_bridge (frb) v2 — RECOMMENDED

Latest stable: flutter_rust_bridge 2.12.0 (Dart pub package, published ~April 2026; prerelease 2.13.0-beta.2 also on pub). docs.rs crate also at 2.12.0. It is an official **Flutter Favorite** (first batch of 7 at the program reboot).

frb is a **codegen** binding generator (`flutter_rust_bridge_codegen generate`): you write plain Rust, annotate, and it emits the Dart FFI glue + a Rust wrapper.

V2 capabilities relevant to a VPN core

- **Arbitrary Rust/Dart types** without manual (de)serialization, even non-Clone / non-serializable types — opaque object handles (`RustOpaque`) cross the bridge.
- **Async Rust (`async fn`)** supported in addition to sync-Rust / async-Dart / sync-Dart. A long-running tunnel supervisor `async fn` is a natural fit.
- **Rust→Dart calls**: Rust can call Dart functions (new in V2; previously only Dart→Rust), useful for callbacks (e.g. ask Dart to (re)acquire a VPN permission).
- **Traits as base classes + trait objects**.
- **New SSE codec** (serialize/deserialize) — "several times faster" than the prior codec under some workloads; deadlock-free auto-locking; improved streams.

Streaming events Rust → Dart (the VPN-status channel)

frb **explicitly supports `StreamSink`**: "Allow StreamSink at any argument" and "Support stream (iterator)". Pattern: a Rust function takes a `StreamSink<T>` argument; Dart receives a `Stream<T>` it can `listen()`/expose. This is the canonical way to push continuous tunnel-state / handshake / bytes-transferred events from a

long-lived Rust task to Dart (the long-standing frb issue #347 "long-lived Rust code that sends data continuously back to Dart" is solved by StreamSink). This `Stream<T>` is exactly what you feed a Riverpod StreamProvider.

Caveat

- Codegen step in the build pipeline (run `frb generate` on Rust API change; commit or regenerate generated Dart/Rust). Bridge crate version MUST match the `flutter_rust_bridge` Dart dep version.

2. Alternative: UniFFI (Mozilla) + uniffi-dart — NOT production-ready for Dart

- **uniffi-rs** is Mozilla's multi-language bindings generator; first-party targets are **Kotlin, Swift, Python, Ruby** (3rd-party C# and Go). Dart is NOT a first-party target.
- **uniffi-dart / uniffi-rs-dart** (NiallBunting) is the community Dart binding. Its own README states it is **a work in progress and "should not be trusted"**, with incomplete foreign-callback and data-type support.
- Verdict (§11.4.6, no-guessing): for a Flutter VPN client, **frb v2 is the mature/ recommended path**; UniFFI-Dart is experimental. UniFFI remains the right call only if the SAME Rust core must also serve native Kotlin/Swift surfaces (e.g. a HarmonyOS ArkTS or platform-channel layer) where UniFFI's first-party Kotlin/Swift output is wanted — then you may run frb for Dart AND UniFFI for Kotlin/Swift from one core. (Note Mullvad's own app uses a Rust daemon talking over gRPC/IPC rather than in-process FFI — an architecture worth weighing vs in-process frb.)

3. Per-platform Rust library packaging (staticlib / cdylib)

Standard Rust-for-Flutter packaging (frb scaffolds most of this via `flutter_rust_bridge_codegen create` / `cargo-ndk` / cargo-xcode tooling):

- **Android**: `crate-type = ["cdylib"]` → `lib<name>.so` per ABI (arm64-v8a, armeabi-v7a, x86_64), built with `cargo-ndk`, bundled in `jniLibs`.
- **iOS / macOS**: `staticlib` (or `cdylib`) → `.a`/`.dylib`, linked as an xcframework; iOS prefers static linkage.
- **Linux / Windows desktop**: `cdylib` → `.so` / `.dll` loaded via Dart FFI `DynamicLibrary`.
- A VPN tunnel typically needs a platform extension process (Android `VpnService`, iOS/macOS `NEPacketTunnelProvider` Network Extension); the Rust core (e.g. a WireGuard/boringtun-style data path) is linked into that extension as a staticlib/cdylib and the FFI control surface is exposed to the Flutter UI process — frb bridges the UI/control side.

4. Flutter on Aurora OS (omprussia/flutter) — current state

- Source: **gitlab.com/omprussia/flutter** (group), with **flutter** (SDK fork) and **flutter-embedder** subprojects, plus a `flutter-community-plugins` group (Aurora-specific plugin forks: pickers_aurora, qr_code_scanner_aurora, flutter_libserialport_aurora, etc.).
- **flutter-embedder** is the Aurora OS runtime/embedder for Flutter apps.
- **Active in 2025**: a "Flutter CLI Implementation for Aurora OS" talk at Mobius 2025 (Autumn) covered extending the Flutter CLI to build/install apps and manage Aurora devices — i.e. the toolchain is maturing but is a SEPARATE fork, not upstream Flutter.
- Implication for the VPN app: Aurora is a SECOND-class/forked target. Plugins must have Aurora variants (the `*_aurora` pattern). FFI to a Rust `.so` is feasible (Aurora is Linux/Qt-based), but every platform plugin (VPN permission, packet tunnel) needs an Aurora-specific implementation — budget this as bespoke work, with honest SKIP where the embedder lacks a needed API.

5. Flutter on HarmonyOS NEXT (OpenHarmony-SIG flutter_flutter) — current state

- Source: **gitee.com/openharmony-sig/flutter_flutter** (the SDK/CLI wrapper), with a `dev` branch; the engine is the **Flutter OHOS branch**. This is the Huawei/OpenHarmony-SIG maintained port enabling Flutter apps on HarmonyOS / OpenHarmony via the `ohos` device channel.
- **Versions (2025)**: the port has tracked specific Flutter baselines — commonly cited: **Flutter OHOS 3.22.x** (and earlier 3.13.9+/3.7.12 custom builds) atop **HarmonyOS SDK 5.0.0(12)** / OpenHarmony **API 10-12**. The Flutter Engine is recompiled for OHOS as the renderer; apps build to `.hap`.
- **Plugin adaptation required**: upstream plugins need OHOS variants (ArkTS/native bridge). Active community work through 2025 (plugin adaptation, native interaction, "existing Flutter project → HarmonyOS" guides).
- Implication for the VPN app: HarmonyOS NEXT is a maintained-but-forked target on a lagging Flutter baseline. Rust `.so` via FFI is feasible (OHOS is Linux-kernel-based), but the VPN tunnel needs HarmonyOS's own VPN extension API (ArkTS `vpnExtension`/`VpnExtensionAbility`) bridged to the Rust core — bespoke per-platform plugin work; honest SKIP where the SIG port lacks the API.

6. Riverpod for the VPN status stream

- **Latest: flutter_riverpod 3.3.2** (pub, published ~June 2026); **Riverpod 3.0** released **September 2025** is the current major line. Flutter Favorite.
- **StreamProvider** is the idiomatic wrapper for a continuous status stream: same behavior as FutureProvider but produces a `Stream` value; "for continuous streams of data (like WebSockets or Firebase)" — here, the frb `Stream<VpnState>`.

- **Riverpod 3.0 behavior to know**:
  - `StreamProvider` now **pauses its `StreamSubscription` when not actively listened** (resource-saving; relevant — a paused subscription means the Rust side may stop being polled when no widget listens).
  - `Stream/FutureProvider.overrideWithValue` was **added back** (useful for tests feeding a fake VPN-state stream → anti-bluff §11.4.27 unit fakes).
  - Use **`.autoDispose`** (or codegen `@riverpod` which is autoDispose by default) for the status stream so the underlying Rust StreamSink/subscription is torn down when no UI listens; a non-autoDispose StreamProvider "is almost never destroyed."
- Recommended shape: Rust `tunnel_events(sink: StreamSink<VpnState>)` → Dart `Stream<VpnState>` → `@riverpod`/`StreamProvider.autoDispose` → UI `ref.watch`.

7. Synthesis / recommendations for the spec

1. **Bridge**: flutter_rust_bridge **2.12.0** (codegen, StreamSink for events) is the primary recommendation; pin Dart-dep version == bridge-crate version.
2. **UniFFI** only if a non-Dart native surface (Kotlin/Swift/HarmonyOS ArkTS) must share the SAME Rust core; uniffi-dart itself is not production-ready.
3. **Packaging**: cdylib (Android/Linux/Windows) + staticlib/xcframework (iOS/macOS); Rust data path linked into the OS VPN extension process.
4. **State**: Riverpod 3.x `StreamProvider.autoDispose` over the frb event stream; account for 3.0 pause-when-unlistened semantics.
5. **Aurora OS & HarmonyOS NEXT**: forked Flutter ports (omprussia/flutter, openharmony-sig/flutter_flutter), both 2025-active but lagging upstream and requiring bespoke per-platform VPN plugins — treat as second-tier targets with honest SKIP-with-reason (§11.4.3) where the embedder/SIG port lacks a needed VPN/permission API; do NOT assume parity with Android/iOS.

Sources verified

- https://pub.dev/packages/flutter_rust_bridge — accessed 2026-06-25 (v2.12.0, StreamSink support, SDK constraints)
- https://cjycode.com/flutter_rust_bridge/guides/miscellaneous/whats-new — accessed 2026-06-25 (V2 features: SSE codec, async fn, traits, Rust→Dart calls)
- https://github.com/fzyzcjy/flutter_rust_bridge — accessed 2026-06-25 (Flutter Favorite, StreamSink/stream support, arbitrary types)
- https://github.com/fzyzcjy/flutter_rust_bridge/issues/347 — accessed 2026-06-25 (long-lived Rust → continuous data to Dart via StreamSink)
- https://docs.rs/crate/flutter_rust_bridge/latest — accessed 2026-06-25 (crate 2.12.0)
- https://github.com/mozilla/uniffi-rs — accessed 2026-06-25 (first-party Kotlin/Swift/Python/Ruby; 3rd-party C#/Go; no first-party Dart)
- https://github.com/NiallBunting/uniffi-rs-dart — accessed 2026-06-25 (Dart binding WIP, "should not be trusted", incomplete callbacks/types)
- https://mozilla.github.io/uniffi-rs/ — accessed 2026-06-25 (UniFFI user guide, supported languages)
- https://gitlab.com/omprussia/flutter — accessed 2026-06-25 (Aurora OS Flutter SDK fork + embedder + community plugins)

- https://gitlab.com/omprussia/flutter/flutter-embedder — accessed 2026-06-25 (Aurora OS Flutter embedder/runtime)
- https://mobiusconf.com/en/talks/26ff8152fa8e4038b85335c874c0b083/ — accessed 2026-06-25 (Flutter CLI for Aurora OS, Mobius 2025 Autumn)
- https://www.harmony-developers.com/p/flutter-app-development-hongmeng — accessed 2026-06-25 (Flutter OHOS 3.13.9+, HarmonyOS SDK 5.0.0(12), OpenHarmony API 10)
- https://dev.to/flfljh/setting-up-flutter-development-environment-for-harmonyos-hik — accessed 2026-06-25 (gitee openharmony-sig/flutter_flutter dev branch, Flutter 3.22 / 3.7.12 OHOS builds)
- https://riverpod.dev/docs/whats_new — accessed 2026-06-25 (Riverpod 3.0 Sept 2025; StreamProvider pause-when-unlistened; overrideWithValue re-added)
- https://pub.dev/packages/flutter_riverpod — accessed 2026-06-25 (flutter_riverpod 3.3.2, 3.x current)
- https://pub.dev/documentation/riverpod/latest/riverpod/StreamProvider-class.html — accessed 2026-06-25 (StreamProvider for continuous streams; autoDispose guidance)